In this section you'll learn how to write unit tests for code that relies on an EF Core `DbContext`. You'll learn how to create an in-memory database, and you'll see the difference between the EF in-memory provider and the SQLite in-memory provider. Finally, you'll see how to use the in-memory SQLite provider to create fast, isolated tests for code that relies on a `DbContext`.

As you saw in chapter 12, EF Core is an object-relational mapper (ORM) that is used primarily with relational databases. In this section I'm going to discuss one way to test services that depend on an EF Core `DbContext` without having to configure or interact with a real database.

NOTE To learn more about testing your EF Core code, see *Entity Framework Core in Action*, 2nd ed., by Jon P. Smith (Manning, 2021), http://mng.bz/QPpR.

The following listing shows a highly stripped-down version of the `RecipeService` you created in chapter 12 for the recipe app. It shows a single method to fetch the details of a recipe using an injected EF Core `DbContext`.

Listing 36.12 `RecipeService` to test, which uses EF Core to store and load entities

```
public class RecipeService
{
    readonly AppDbContext _context;                ❶
    public RecipeService(AppDbContext context)     ❶
    {                                              ❶
        _context = context;                        ❶
    }                                              ❶
    public RecipeViewModel GetRecipe(int id)
    {
        return _context.Recipes                    ❷
            .Where(x => x.RecipeId == id)
```